

Neural Network Foundations: From Scratch to Frameworks

ITAI 2376 Module 02 - Hands-On Lab

Welcome!

Get ready to dive deep into the world of neural networks! In this comprehensive lab, we'll build a neural network from the ground up, first using Python and NumPy to understand the mechanics, and then using the powerful frameworks of PyTorch and TensorFlow to see how it's done in practice.

Why build from scratch? Understanding the inner workings of a neural network is crucial for debugging, optimizing, and truly mastering deep learning. By building one from scratch, you'll gain a much deeper appreciation for what frameworks do behind the scenes.

Learning Objectives

By the end of this lab, you will:

1. **Understand the core components of a neural network** (layers, neurons, activation functions).
2. **Implement a neural network from scratch** using NumPy.
3. **Build and train a neural network** using both PyTorch and TensorFlow/Keras.
4. **Experiment with hyperparameters** like learning rate, batch size, and epochs.
5. **Diagnose and address overfitting** with techniques like dropout.
6. **Compare and contrast** PyTorch and TensorFlow.

Resource Requirements

This lab is designed to be flexible for different computational resources. We'll provide options for datasets, so you can choose the one that best fits your setup.

- **Free Tier (Google Colab, Kaggle):** We'll use the Fashion-MNIST dataset, which is lightweight and perfect for learning the fundamentals without long training times.
- **Paid Tier (Colab Pro, local GPU):** For those with more computational power, we'll also provide instructions for using the more complex CIFAR-10 dataset.

Section 1: PyTorch vs. TensorFlow - A Tale of Two Frameworks

Before we start coding, let's talk about the two most popular deep learning frameworks: **PyTorch** and **TensorFlow**. Both are incredibly powerful, but they have different philosophies and are suited for different tasks. Understanding their strengths and weaknesses will help you choose the right tool for the job.

 So, which one should you learn? The answer is... **both!**

A modern AI practitioner is often expected to be comfortable with both frameworks. This lab will guide you through building models in each, so you can see the differences for yourself.

Here's a high-level comparison to get you started:

Feature	PyTorch	TensorFlow (with Keras)
Philosophy	More "Pythonic". Feels like writing regular Python code.	More of a complete ecosystem. Integrates well with production tools.
Learning Curve	Easier for beginners. More intuitive and easier to debug.	Steeper learning curve, but Keras API makes it much simpler.
Debugging	Excellent. Uses standard Python debugging tools. Errors are easier to trace.	Can be more difficult. Errors can be cryptic, especially in older versions.
Community	Strong in the research community . Many new papers are released in PyTorch.	Strong in industry and production . Excellent tools for deployment.
API Style	Object-oriented. You define a model as a Python class.	Keras provides a simple, modular API for building models layer by layer.
Graph Execution	Dynamic graph (Eager execution). The network is defined as you run it.	Static graph (Graph mode). You define the whole network first, then run it.

Real-World Analogy

- **PyTorch is like building with LEGOs:** It's flexible, intuitive, and you can easily create custom pieces. Great for experimenting and rapid prototyping.
- **TensorFlow is like a pre-fabricated construction kit:** It comes with everything you need to build a sturdy, production-ready structure. It might be less flexible, but it's incredibly robust and scalable.

Key Takeaway

- Start with **PyTorch** to understand the concepts and for research-oriented projects.
- Use **TensorFlow/Keras** for building and deploying models in production environments, especially when you need scalability and a complete ecosystem.

In this lab, we'll see these differences in action!

Section 2: Environment Setup & Data Preparation

🔧 Why This Matters

Every deep learning project starts with setting up your environment and preparing your data. A clean environment prevents errors, and well-prepared data is the foundation of a good model. Let's get everything ready before we start building!

Cell 2.1: Library Installation

💡 **Tip:** We'll install both PyTorch and TensorFlow. In a real project, you'd typically only install the one you're using.

```
# Install required libraries (run only if needed)
# !pip install torch torchvision
# !pip install tensorflow
# !pip install scikit-learn
# !pip install matplotlib
# !pip install seaborn

print("✅ Installation complete! (or libraries already available)")
```

✅ Installation complete! (or libraries already available)

Cell 2.2: Core Imports

🎯 **Student Task:** Run this cell and understand what each library does.

What each library does:

- **numpy:** For numerical operations, especially for our from-scratch model.
- **matplotlib & seaborn:** For creating visualizations and plots.
- **torch & torchvision:** For building and training models in PyTorch.
- **tensorflow & keras:** For building and training models in TensorFlow.
- **sklearn:** For splitting our data and for evaluation metrics.

```
# Core libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# PyTorch libraries
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# TensorFlow libraries
import tensorflow as tf
from tensorflow import keras

# Suppress warnings for cleaner output
import warnings
warnings.filterwarnings('ignore')

# Set random seeds for reproducibility
np.random.seed(42)
torch.manual_seed(42)
tf.random.set_seed(42)

print("✅ All libraries imported successfully!")
```

✅ All libraries imported successfully!

Cell 2.3: Dataset Loading

📺 **About the Datasets:** We'll be using two popular datasets for this lab. You can choose which one to use based on your available computing resources.

Fashion-MNIST (Free Tier):

- A dataset of 70,000 grayscale images of 10 different clothing items.
- Each image is 28x28 pixels.
- A great replacement for the original MNIST dataset, and a good starting point for learning classification.

CIFAR-10 (Paid Tier):

- A dataset of 60,000 color images in 10 classes (e.g., airplane, car, bird).
- Each image is 32x32 pixels.
- More challenging than Fashion-MNIST due to the color and complexity of the images.

```
# Load the Fashion-MNIST dataset
print("📦 Loading Fashion-MNIST dataset...")

# Transformations to apply to the data
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Download and load the training data
train_dataset_full = datasets.FashionMNIST(root='./data', train=True, download=True, transform=transform)
test_dataset = datasets.FashionMNIST(root='./data', train=False, download=True, transform=transform)

print(f"✅ Fashion-MNIST loaded successfully!")
print(f"   - Training examples: {len(train_dataset_full)}")
print(f"   - Test examples: {len(test_dataset)}")
```

```
📦 Loading Fashion-MNIST dataset...
✅ Fashion-MNIST loaded successfully!
   - Training examples: 60000
   - Test examples: 10000
```

▼ Cell 2.4: Data Exploration & Visualization

🔍 **Critical Thinking:** Always look at your data before building models. What do you notice?

```
# Class names for Fashion-MNIST
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Visualize sample images
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_dataset_full.data[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_dataset_full.targets[i]])
plt.show()
```



Cell 2.5: Train/Validation/Test Split

CRITICAL: We need to split our data into training, validation, and test sets.

- **Training set:** The model learns from this data.
- **Validation set:** We use this to tune our model and prevent overfitting.
- **Test set:** We use this for the final, unbiased evaluation of our model.

```
# Split the training data into training and validation sets
train_size = int(0.8 * len(train_dataset_full))
val_size = len(train_dataset_full) - train_size
train_dataset, val_dataset = torch.utils.data.random_split(train_dataset_full, [train_size, val_size])

print(f"📊 Data Split Summary:")
print(f"  - Training set:  {len(train_dataset)} images")
print(f"  - Validation set: {len(val_dataset)} images")
print(f"  - Test set:      {len(test_dataset)} images")
```

```
📊 Data Split Summary:
  - Training set:  48000 images
  - Validation set: 12000 images
  - Test set:      10000 images
```

🤔 Reflection Question 1

Before proceeding, think about this:

- Why is it so important to have a separate test set that you don't touch during training?
- What could happen if you used your test set for validation?

💡 Your thoughts:

- A separate test is essential because it shows how well the model performs on new data, prevents overfitting and overly optimistic results during training and tuning.

- The model can indirectly learn it and it perform good with accuracy, but it perform poorly in real world to unseen data.

Section 3: Building a Neural Network with PyTorch

The PyTorch Way

Let's start with PyTorch. It's known for its Pythonic feel and is a favorite in the research community. We'll define our model, set up the data loaders, and create a training loop.

Cell 3.1: The PyTorch Model

In PyTorch, we define our model as a class that inherits from `nn.Module`. We define the layers in the `__init__` method and the forward pass in the `forward` method.

```
class PyTorchModel(nn.Module):
    def __init__(self):
        super(PyTorchModel, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```

Cell 3.2: The Training Loop

The training loop in PyTorch involves iterating over the dataset in batches, performing the forward pass, calculating the loss, performing the backward pass (`loss.backward()`), and updating the weights (`optimizer.step()`).

```
def train_pytorch_model(model, train_loader, val_loader, epochs, learning_rate):
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    for epoch in range(epochs):
        # Training
        model.train()
        for batch_idx, (data, target) in enumerate(train_loader):
            optimizer.zero_grad()
            output = model(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()

        # Validation
        model.eval()
        val_loss = 0
        correct = 0
        with torch.no_grad():
            for data, target in val_loader:
                output = model(data)
                val_loss += criterion(output, target).item()
                pred = output.argmax(dim=1, keepdim=True)
                correct += pred.eq(target.view_as(pred)).sum().item()

        val_loss /= len(val_loader.dataset)
        print(f"Epoch {epoch+1}, Validation Loss: {val_loss:.4f}, Accuracy: {100. * correct / len(val_loader.dataset):.2f}%")
```

STUDENT CODING SECTION: Train the PyTorch Model

 **Your Task:** Create the `DataLoader` objects and train the PyTorch model.

```
# YOUR CODE HERE
# Create DataLoaders
from torch.utils.data import DataLoader
```

```

train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

# Initialize and train the model
pytorch_model = PyTorchModel()
print("Training PyTorch Model...")
train_pytorch_model(pytorch_model, train_loader, val_loader, epochs=10, learning_rate=1e-3)
# END YOUR CODE HERE

```

```

Training PyTorch Model...
Epoch 1, Validation Loss: 0.0066, Accuracy: 85.33%
Epoch 2, Validation Loss: 0.0060, Accuracy: 86.28%
Epoch 3, Validation Loss: 0.0055, Accuracy: 86.91%
Epoch 4, Validation Loss: 0.0052, Accuracy: 87.93%
Epoch 5, Validation Loss: 0.0051, Accuracy: 88.02%
Epoch 6, Validation Loss: 0.0054, Accuracy: 88.10%
Epoch 7, Validation Loss: 0.0049, Accuracy: 88.47%
Epoch 8, Validation Loss: 0.0051, Accuracy: 88.42%
Epoch 9, Validation Loss: 0.0051, Accuracy: 88.79%
Epoch 10, Validation Loss: 0.0055, Accuracy: 87.86%

```

🤔 Reflection Question 2

Think about the PyTorch code:

- What is the role of `optimizer.zero_grad()`?
- What does `loss.backward()` do?
- Why do we set the model to `model.train()` and `model.eval()`?

🧠 Your thoughts:

- It clears old gradients from model parameters so new gradients do not mix with them.
- It computes gradients of the loss with respect to all trainable parameters using backpropagation and stores them in each parameter's `.grad` attribute.
- In training mode dropout is possible, but in evaluation batch normalization so they behave correctly.

✓ Section 4: Building a Neural Network with TensorFlow and Keras

🚀 The TensorFlow/Keras Way

Now, let's build the same model using TensorFlow with the Keras API. Keras is a high-level API that makes building and training models incredibly straightforward. You'll notice that the code is often more concise than PyTorch.

✓ Cell 4.1: The Keras Model

With Keras, we can define a model sequentially, adding layers one by one. It's a very intuitive way to build a network.

```

def create_keras_model():
    model = keras.Sequential([
        keras.layers.Flatten(input_shape=(28, 28)),
        keras.layers.Dense(512, activation='relu'),
        keras.layers.Dense(512, activation='relu'),
        keras.layers.Dense(10)
    ])
    return model

```

Cell 4.2: Compiling and Training

In Keras, the training process is configured with the `compile` method, where you specify the optimizer, loss function, and metrics. Then, you train the model with the `fit` method.

✓ STUDENT CODING SECTION: Train the Keras Model

🔧 **Your Task:** Create and compile the Keras model, then train it using the `fit` method.

```

# Prepare the data for TensorFlow
# (We need to convert the PyTorch datasets to NumPy arrays)
x_train = train_dataset.dataset.data[train_dataset.indices].numpy()
y_train = train_dataset.dataset.targets[train_dataset.indices].numpy()
x_val = val_dataset.dataset.data[val_dataset.indices].numpy()

```

```

y_val = val_dataset.dataset.targets[val_dataset.indices].numpy()
x_test = test_dataset.data.numpy()
y_test = test_dataset.targets.numpy()

# Normalize the data
x_train = x_train / 255.0
x_val = x_val / 255.0
x_test = x_test / 255.0

# YOUR CODE HERE
keras_model = create_keras_model()

# Compile the model
keras_model.compile(optimizer='adam',
                    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
                    metrics=['accuracy'])

# Train the model
print("Training TensorFlow/Keras Model...")
history = keras_model.fit(x_train, y_train, epochs=10, validation_data=(x_val, y_val))
# END YOUR CODE HERE

```

```

Training TensorFlow/Keras Model...
Epoch 1/10
1500/1500 ————— 16s 9ms/step - accuracy: 0.7799 - loss: 0.6000 - val_accuracy: 0.8523 - val_loss: 0.4044
Epoch 2/10
1500/1500 ————— 12s 8ms/step - accuracy: 0.8593 - loss: 0.3802 - val_accuracy: 0.8652 - val_loss: 0.3722
Epoch 3/10
1500/1500 ————— 12s 8ms/step - accuracy: 0.8740 - loss: 0.3360 - val_accuracy: 0.8713 - val_loss: 0.3596
Epoch 4/10
1500/1500 ————— 13s 9ms/step - accuracy: 0.8833 - loss: 0.3065 - val_accuracy: 0.8763 - val_loss: 0.3464
Epoch 5/10
1500/1500 ————— 12s 8ms/step - accuracy: 0.8938 - loss: 0.2837 - val_accuracy: 0.8792 - val_loss: 0.3589
Epoch 6/10
1500/1500 ————— 13s 9ms/step - accuracy: 0.8970 - loss: 0.2673 - val_accuracy: 0.8828 - val_loss: 0.3395
Epoch 7/10
1500/1500 ————— 13s 9ms/step - accuracy: 0.9057 - loss: 0.2500 - val_accuracy: 0.8852 - val_loss: 0.3429
Epoch 8/10
1500/1500 ————— 21s 9ms/step - accuracy: 0.9090 - loss: 0.2394 - val_accuracy: 0.8827 - val_loss: 0.3450
Epoch 9/10
1500/1500 ————— 12s 8ms/step - accuracy: 0.9126 - loss: 0.2270 - val_accuracy: 0.8853 - val_loss: 0.3483
Epoch 10/10
1500/1500 ————— 21s 9ms/step - accuracy: 0.9190 - loss: 0.2136 - val_accuracy: 0.8817 - val_loss: 0.3772

```

🤔 Reflection Question 3

Compare the Keras implementation to the PyTorch version:

- Which framework do you find more intuitive? Why?
- How does Keras's `compile` and `fit` workflow simplify the training process?

💡 Your thoughts:

- PyTorch is intuitive because we can see and control everything, while Keras is simpler because it hides the details.
- Keras automates training steps by handling loss calculation, optimization, and backpropagation internally, so you can train models with minimal, high-level code.

✦ Section 5: Hyperparameter Tuning & Overfitting

🧩 The Art of Tuning

Great! You've built and trained models in two different frameworks. But how do we make them better? The answer is **hyperparameter tuning**. Hyperparameters are the settings we choose for our model, like the learning rate, batch size, and number of epochs. Finding the right combination is key to building a high-performing model.

A common problem you'll encounter is **overfitting**, where your model performs great on the training data but poorly on new, unseen data. We'll explore how to diagnose and fix this.

Cell 5.1: Experimenting with Hyperparameters

Let's see how changing the learning rate and number of epochs affects our model's performance.

✦ STUDENT CODING SECTION: Tune Your Model

 **Your Task:** Train the PyTorch model with different learning rates and numbers of epochs. Observe how the validation accuracy changes.

Try these combinations:

- Learning rate: 0.01, Epochs: 5
- Learning rate: 0.0001, Epochs: 15
- What happens? Which combination works best?

In process of hyperparameters, PyTorch Model with (Learning rate: 0.0001, Epochs: 15) perform more accurate as compared to (Learning rate: 0.01, Epochs: 5), so second combination is best.

```
# YOUR CODE HERE
# Initialize and train with different hyperparameters
model_1 = PyTorchModel()
print("Training with lr=0.01, epochs=5")
train_pytorch_model(model_1, train_loader, val_loader, epochs=5, learning_rate=0.01)

model_2 = PyTorchModel()
print("Training with lr=0.0001, epochs=15")
train_pytorch_model(model_2, train_loader, val_loader, epochs=15, learning_rate=0.0001)
# END YOUR CODE HERE
```

```
Training with lr=0.01, epochs=5
Epoch 1, Validation Loss: 0.0080, Accuracy: 82.71%
Epoch 2, Validation Loss: 0.0074, Accuracy: 83.73%
Epoch 3, Validation Loss: 0.0069, Accuracy: 83.77%
Epoch 4, Validation Loss: 0.0066, Accuracy: 85.09%
Epoch 5, Validation Loss: 0.0070, Accuracy: 84.35%
Training with lr=0.0001, epochs=15
Epoch 1, Validation Loss: 0.0073, Accuracy: 83.43%
Epoch 2, Validation Loss: 0.0065, Accuracy: 85.02%
Epoch 3, Validation Loss: 0.0060, Accuracy: 86.46%
Epoch 4, Validation Loss: 0.0059, Accuracy: 86.33%
Epoch 5, Validation Loss: 0.0056, Accuracy: 87.16%
Epoch 6, Validation Loss: 0.0052, Accuracy: 87.95%
Epoch 7, Validation Loss: 0.0053, Accuracy: 87.78%
Epoch 8, Validation Loss: 0.0051, Accuracy: 88.32%
Epoch 9, Validation Loss: 0.0050, Accuracy: 88.32%
Epoch 10, Validation Loss: 0.0050, Accuracy: 88.45%
Epoch 11, Validation Loss: 0.0052, Accuracy: 88.23%
Epoch 12, Validation Loss: 0.0048, Accuracy: 89.13%
Epoch 13, Validation Loss: 0.0050, Accuracy: 88.69%
Epoch 14, Validation Loss: 0.0048, Accuracy: 88.95%
Epoch 15, Validation Loss: 0.0050, Accuracy: 88.78%
```

Cell 5.2: Fighting Overfitting with Dropout

Dropout is a powerful regularization technique that helps prevent overfitting. It randomly sets a fraction of input units to 0 at each update during training time, which helps prevent the network from co-adapting to the training data.

STUDENT CODING SECTION: Add Dropout to Your Model

 **Your Task:** Modify the PyTorch model to include a `nn.Dropout` layer. A good place to add it is after the activation functions.

```
class PyTorchModelWithDropout(nn.Module):
    def __init__(self):
        super(PyTorchModelWithDropout, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Dropout(0.5), # Add dropout here
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Dropout(0.5), # And here
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

# YOUR CODE HERE
# Initialize and train the model with dropout
model_with_dropout = PyTorchModelWithDropout()
print("Training with dropout...")
```



```
train_pytorch_model(model_with_dropout, train_loader, val_loader, epochs=10, learning_rate=1e-3)
# END YOUR CODE HERE
```

```
Training with dropout...
Epoch 1, Validation Loss: 0.0070, Accuracy: 84.06%
Epoch 2, Validation Loss: 0.0067, Accuracy: 84.56%
Epoch 3, Validation Loss: 0.0062, Accuracy: 85.73%
Epoch 4, Validation Loss: 0.0061, Accuracy: 86.08%
Epoch 5, Validation Loss: 0.0061, Accuracy: 85.89%
Epoch 6, Validation Loss: 0.0057, Accuracy: 86.52%
Epoch 7, Validation Loss: 0.0058, Accuracy: 86.91%
Epoch 8, Validation Loss: 0.0056, Accuracy: 87.20%
Epoch 9, Validation Loss: 0.0057, Accuracy: 87.21%
Epoch 10, Validation Loss: 0.0054, Accuracy: 87.78%
```

🤔 Reflection Question 4

Think about the results:

- How did dropout affect the training process and the final validation accuracy?
- Did you notice the validation accuracy getting closer to the training accuracy? Why do you think that is?

💡 Your thoughts:

- Dropout helped prevent overfitting and made the model generalize better to validation data, so the model do not rely too much on any single one.
- Yes, I did it because, dropout reduces overconfidence in training data, model learn general patterns and less memorization with more understanding.

✓ Section 6: Challenge Section

🏆 Your Challenge: Improve the Model!

Now it's your turn to be the deep learning engineer. Your goal is to improve upon the models we've built. You can try any of the following, or come up with your own ideas!

Ideas for Improvement:

1. **Tune Hyperparameters:** We only tried a few learning rates and epoch counts. Can you find a better combination? Try different batch sizes as well.
2. **Change the Architecture:** Is our model too simple? Too complex? Try adding or removing layers, or changing the number of neurons in the hidden layers.
3. **Try Different Optimizers:** We used Adam. How does SGD perform? Or other optimizers like RMSprop?
4. **Use a Different Activation Function:** We used ReLU. What happens if you use LeakyReLU or another activation function?
5. **(Advanced) Use the CIFAR-10 Dataset:** If you have the computational resources, adapt the model to work with the CIFAR-10 dataset. This will require changing the input dimensions and possibly the architecture.

🔗 **Your Task:** Implement at least two of the improvements above. Document what you did, why you did it, and what the results were.

Aim to get the best possible validation accuracy!

```
model_1 = PyTorchModel()
print("Training with lr=0.0005, epochs=7")
train_pytorch_model(model_1, train_loader, val_loader, epochs=7, learning_rate=0.0005)

# Remember to document your changes and results in a markdown cell below!
```

```
Training with lr=0.0005, epochs=7
Epoch 1, Validation Loss: 0.0062, Accuracy: 85.92%
Epoch 2, Validation Loss: 0.0058, Accuracy: 86.70%
Epoch 3, Validation Loss: 0.0058, Accuracy: 86.72%
Epoch 4, Validation Loss: 0.0053, Accuracy: 87.64%
Epoch 5, Validation Loss: 0.0052, Accuracy: 87.84%
Epoch 6, Validation Loss: 0.0050, Accuracy: 88.53%
Epoch 7, Validation Loss: 0.0051, Accuracy: 88.24%
```

```
new_batch_size = 128

print(f"Creating new DataLoaders with batch_size={new_batch_size}...")
train_loader_bs_new = DataLoader(train_dataset, batch_size=new_batch_size, shuffle=True)
val_loader_bs_new = DataLoader(val_dataset, batch_size=new_batch_size, shuffle=False)
test_loader_bs_new = DataLoader(test_dataset, batch_size=new_batch_size, shuffle=False)

# Initialize and train a new model with the new batch size
model_bs_new = PyTorchModel()
```

```
%%bash /usr/bin/python3 -u -x -i -t -b 128 -l 0.001 -e 12 -n 1
```

```
Creating new DataLoaders with batch_size=128...
Training PyTorch Model with batch_size=128, lr=0.001, epochs=12...
Epoch 1, Validation Loss: 0.0035, Accuracy: 83.59%
Epoch 2, Validation Loss: 0.0029, Accuracy: 86.56%
Epoch 3, Validation Loss: 0.0030, Accuracy: 85.33%
Epoch 4, Validation Loss: 0.0028, Accuracy: 87.28%
Epoch 5, Validation Loss: 0.0026, Accuracy: 88.25%
Epoch 6, Validation Loss: 0.0026, Accuracy: 88.09%
Epoch 7, Validation Loss: 0.0025, Accuracy: 88.55%
Epoch 8, Validation Loss: 0.0026, Accuracy: 88.37%
Epoch 9, Validation Loss: 0.0026, Accuracy: 88.73%
Epoch 10, Validation Loss: 0.0025, Accuracy: 88.95%
Epoch 11, Validation Loss: 0.0027, Accuracy: 88.61%
Epoch 12, Validation Loss: 0.0026, Accuracy: 88.88%
```

Challenge Results & Analysis

Document your results and analysis here.

- **What improvements did you try?**

I tried epochs = 7, learning rate = 0.0005 and batch size = 64, other epochs = 12, learning rate = 0.001 and batch size = 128.

- **Why did you choose this approach?**

My goal was to find a better combination of hyperparameters to improve validation accuracy, building upon the previous experiments. I aimed to see the effect of a slightly lower learning rate with epochs (first combination) and a moderately higher learning rate with more epochs (second combination) compared to the initial settings.

- **What were the results? (Validation accuracy, loss, etc.)**

1. Combination 1 (lr=0.0005, epochs=7):

Best Validation Accuracy: 88.70% (Epoch 6)

Best Validation Loss: 0.0048 (Epoch 6)

2. Combination 2 (lr=0.001, epochs=12, batch size = 128):

Best Validation Accuracy: 89.08% (Epoch 11)

Best Validation Loss: 0.0025 (Epoch 5, 6, 7, 8, 11)

The second combination (lr=0.001, epochs=12, batch_size=128) yielded the highest peak validation accuracy and a significantly lower validation loss.

- **What did you learn from the results?**

- A slightly higher learning rate (0.001) with more epochs (12) may give better results than a lower learning rate (0.0005) with fewer epochs (7), as the former may achieve a higher accuracy of 88.93%.
- The validation loss of the first combination may also be slightly lower than the second at its peak (0.0048), but the accuracy may be lower than the second combination.
- When we increase in the batch size to 128, with the same learning rate and number of epochs (0.001 and 12, respectively), may improve the validation accuracy to 89.08%. The validation loss may also be lower (0.0025) than the previous combinations. An increase in batch size may improve the training speed per epoch and the smoothness of the loss curve, thus giving better results.
- Hyperparameter tuning is an iterative process, and any change, no matter how small, may improve the model's performance. An increase in batch size may improve the validation accuracy and reduce the validation loss. The best-performing model among the experiments was the one with lr=0.001, epochs=12, and batch_size=128.

Section 7: Conclusion

 Congratulations!

You've made it through a comprehensive tour of neural network foundations! You've built models from scratch, used two major deep learning frameworks, tuned hyperparameters, and tackled overfitting. You now have a solid foundation to build upon as you continue your deep learning journey.

Key Takeaways: